MK-1: an exactly simulated two-qubit ZZ-feature-map fidelity kernel, with machine-checked kernel properties and a finite-shot hardware check

Benjamin Stanley Frohman
ORCID 0009-0006-7068-3718

November 3, 2026; revised November 4, 2026

Abstract

MK-1 is a small, self-contained implementation of a quantum fidelity kernel on two qubits: a ZZ-type feature map $U(x)$, the compute–uncompute estimator $K(x, x') = |\langle 00|U(x')^\dagger U(x)|00\rangle|^2$, and a visualisation of the kernel-SVM pipeline. The construction is not new; it follows Havlíček et al. [1] and Schuld–Killoran [2]. This note records exactly what the code computes, documents a defect in the original script (its "entangling" gate was a no-op, so the ~~kernel factorised~~state is a product state and the kernel factorises, proved in Lean on explicit 4×4 matrices), and lists what is machine-checked in Lean 4 with Mathlib: for any unitary feature map, $K(x, x) = 1$, symmetry, $0 \le K \le 1$, compute–uncompute equals state fidelity, and positive semidefiniteness of the Gram matrix. A small finite-shot study (6 circuits, $N = 256$–8192) on a simulator and on one IBM device is reported. No quantum advantage ~~, hardware result~~ or new learning-theoretic claim is made, and no SVM is trained.

1 The construction

Qubit 0 is the first tensor factor and the CNOT control. For $x = (x_0, x_1) \in \mathbb{R}^2$,

$$U(x) = \mathrm{CNOT}\,(I \otimes R_Z(2x_0x_1))\,\mathrm{CNOT}\,(R_Z(2x_0) \otimes R_Z(2x_1))\,(H \otimes H), \qquad R_Z(\theta) = \mathrm{diag}(e^{-i\theta/2}, e^{i\theta/2}).$$

Since $\mathrm{CNOT}(I \otimes R_Z(\theta))\mathrm{CNOT} = e^{-i\theta Z\otimes Z/2}$, writing basis states by signs $s \in \{\pm 1\}^2$,

$$U(x)|00\rangle = \tfrac{1}{2}\sum_s e^{-i\varphi(s,x)}|s\rangle, \qquad \varphi(s, x) = x_0s_0 + x_1s_1 + x_0x_1s_0s_1,$$

so with $\Delta = x - x'$ and $\Delta_{01} = x_0x_1 - x'_0x'_1$,

$$K(x, x') = \left|\tfrac{1}{4}\sum_s e^{-i(\Delta_0 s_0 + \Delta_1 s_1 + \Delta_{01} s_0 s_1)}\right|^2.$$

This is one layer of the ZZ feature map of [1] with the simplified pair coefficient x_0x_1 (their map uses $(\pi - x_0)(\pi - x_1)$ and two layers). With one layer, and on two qubits, the kernel has the closed form above and is trivially classically computable; MK-1 is a teaching and testing artefact. The closed form (`kernel_closed_form`) agrees with the statevector kernel to 10^{-12} in the tests (1.3×10^{-15} on the 16 pairs of Section 5). General results on when quantum kernels concentrate or can be dequantised [3, 4, 5] are cited, not proved or formalised here.

2 A defect in the original script

The original `mk1_moving_tower.py` (kept in `original/`) applied the middle $R_Z(2x_0x_1)$ to the *control* qubit. A diagonal gate on the control commutes with CNOT, so the two CNOTs cancel and no $Z \otimes Z$ phase is produced; the kernel becomes the product $\cos^2(\Delta_0 + \Delta_{01}) \cos^2 \Delta_1$. The package keeps this as `variant="legacy"` for reproducibility and uses the corrected map by default. Both facts (CNOT$(D \otimes I)$CNOT $= D \otimes I$ and the $Z \otimes Z$ phase for a target gate) are proved in Lean (`cnot_onControl_cnot`, also stated as `phantom_entangler`; `cnot_onTarget_cnot`). `lean/MK1/Factorization.le` proves on explicit 4×4 complex matrices that the legacy state is a product state (`legacyState_eq_prod`), that inner products and kernels of product states factor (`inner_prod2`, `kernel4_prod2`), hence the legacy kernel is a product of single-qubit kernels (`legacy_kernel_factors`); it also proves `kernel4_symm` and `kernel4_self` for the concrete 4-dimensional kernel.

3 What is machine-checked

`lean/MK1/FidelityKernel.lean` defines $K_\psi(x, y) = |\langle \psi(y), \psi(x) \rangle|^2$ for any family ψ in a complex inner product space and proves:

- `fidelity_self`, `fidelity_symm`, `fidelity_nonneg`, `fidelity_le_one`: $K(x, x) = 1$ for unit states, symmetry, $0 \le K \le 1$ (Cauchy–Schwarz);
- `norm_featureState`, `computeUncompute_eq_fidelity`: states $U(x)v$ from unitaries (linear isometry equivalences) are unit vectors, and $|\langle v, U(y)^{-1}U(x)v \rangle|^2 = K(x, y)$;
- `fidelity_gram_posSemidef`: for states in $\mathbb{C}^n$ and finitely many points, the matrix $[K(x_j, x_i)]$ is positive semidefinite, via $|\langle a, b \rangle|^2 = \langle a \otimes \bar{a}, b \otimes \bar{b} \rangle$ and Mathlib's Gram-matrix theorem.

All are `sorry`-free; `AxiomAudit.lean` shows only `propext`, `Classical.choice`, `Quot.sound`; CI rebuilds and audits on every push. These are standard facts; the Lean development checks them, it does not establish anything new. The Lean code does not model the specific gate angles beyond the CNOT identities and the legacy factorisation; the agreement of the numerical circuit with the closed form is checked by the Python tests.

4 What is tested numerically

The Python tests (CI, every push) check for both variants: unitarity of $U(x)$; $K(x, x) = 1$; symmetry; $0 \le K \le 1$; compute–uncompute equals $|\langle \psi(x') | \psi(x) \rangle|^2$; agreement with the matrix-free closed form to 10^{-12}; Gram matrices symmetric with minimum eigenvalue $> -10^{-10}$; and the two CNOT identities.

5 ~~Limitations~~Finite-shot estimation

~~Exact statevector simulation, no shot noise or hardware ; two qubits;~~ For four points (seed 7) the exact Gram matrix has off-diagonal entries 0.1246, 0.0840, 0.2525, 0.9176, 0.3283, 0.3298 (minimum eigenvalue 0.081). Each is estimated as the frequency of 00 in a compute–uncompute circuit, using the first N of 8192 shots. RMS error over the six entries:

N	256	512	1024	2048	4096	8192	log–log slope
Aer simulator (seed 11)	.0234	.0105	.0084	.0102	.0045	.0044	−0.44
IBM `ibm_kingston`	.0373	.0330	.0299	.0218	.0213	.0194	−0.20

A 400-repetition binomial study gives slope -0.506, consistent with $N^{-1/2}$. On hardware the error levels off near 0.02: device noise adds a bias that more shots do not remove. Run: `ibm_kingston`, 2026-11-04 20:42 UTC, one SamplerV2 job `dblpgrd8ft8s738ths20`, 6 circuits, 8192 shots, transpiled depth 18, no error mitigation (`hardware/run.py`, `results/`).

6 Limitations

Two qubits; one hardware job without mitigation; no SVM is trained in the package (the DUAL/DECIDE stages are illustrative); no claim of quantum advantage. For the limits of such kernels see [3].

References

[1] V. Havlíček, A. D. Córcoles, K. Temme, A. W. Harrow, A. Kandala, J. M. Chow, J. M. Gambetta, Supervised learning with quantum-enhanced feature spaces, *Nature* 567 (2019) 209–212, doi:10.1038/s41586-019-0980-2.

[2] M. Schuld, N. Killoran, Quantum machine learning in feature Hilbert spaces, *Phys. Rev. Lett.* 122 (2019) 040504.

[3] J. M. Kübler, S. Buchholz, B. Schölkopf, The inductive bias of quantum kernels, *NeurIPS* 34 (2021).

[4] S. Thanasilp, S. Wang, M. Cerezo, Z. Holmes, Exponential concentration in quantum kernel methods, *Nat. Commun.* 15 (2024) 5200.

[5] H.-Y. Huang et al., Power of data in quantum machine learning, *Nat. Commun.* 12 (2021) 2631.

Code: `https://github.com/BenFrohman/mk1-kernel-limits`. License: Apache-2.0 (code), CC BY 4.0 (this note).